



TRƯỜNG ĐẠI HỌC QUY NHƠN

TIỂU LUẬN HỌC PHẦN
LÝ THUYẾT TỐI ƯU

THUẬT TOÁN DI TRUYỀN:
CƠ SỞ LÝ THUYẾT VÀ CÁC
ỨNG DỤNG

Trình độ đào tạo:	Thạc sĩ
Học viên thực hiện:	Nguyễn Hồ Bảo Thiên
Lớp:	Khoa học dữ liệu K28B
Giảng viên hướng dẫn:	TS. Nguyễn Văn Vũ

Gia Lai, 2026

Mục lục

1	Cơ sở lý thuyết về Thuật toán Di truyền	4
1.1	Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa	4
1.2	Cơ sở Khoa học và Nguyên lý Hoạt động của GA	6
1.3	Các Thành phần và Phép toán Cốt lõi	7
1.4	Quy trình Thuật toán và Điều kiện Dừng	11
1.5	Lập trình Di truyền (Genetic Programming)	13
2	Ứng dụng: Thực nghiệm trên Bốn Bài toán	15
2.1	Tối ưu không ràng buộc: So sánh GA với PyGAD	15
2.2	Tối ưu có ràng buộc: So sánh GA với SciPy SLSQP	17
2.3	Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ	19
2.4	Hồi quy Ký hiệu bằng Lập trình Di truyền	20
2.5	Tổng kết Chương	22
	Tài liệu tham khảo	23

Danh sách hình vẽ

1.1	Chu trình tiến hóa của Thuật toán Di truyền	12
-----	---	----

Danh sách bảng

1.1	Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền	7
2.1	Kết quả GA và PyGAD trên năm hàm benchmark không ràng buộc . .	17
2.2	Kết quả GA và SciPy SLSQP trên năm bài toán tối ưu có ràng buộc .	19
2.3	Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95	20
2.4	Kết quả Hồi quy Tuyến tính và GP trên hai đại lượng vật lý phái sinh	21

Chương 1

Cơ sở lý thuyết về Thuật toán Di truyền

1.1 Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa

Bài toán Tối ưu hóa và Không gian Tìm kiếm

Một bài toán tối ưu hóa tổng quát yêu cầu tìm một vector biến quyết định $x = (x_1, x_2, \dots, x_n)$ sao cho một hàm mục tiêu (objective function) $f(x)$ đạt giá trị cực tiểu hoặc cực đại, trong khi x vẫn thỏa mãn một tập ràng buộc xác định miền khả thi (feasible region) Ω :

$$\min_{x \in \Omega} f(x) \quad \text{hoặc} \quad \max_{x \in \Omega} f(x). \quad (1.1)$$

Miền $\Omega \subseteq \mathbb{R}^n$, còn gọi là không gian tìm kiếm (search space), thường được xác định bởi một hệ ràng buộc đẳng thức và bất đẳng thức trên các biến quyết định. Bài toán được coi là **có ràng buộc** (constrained) khi Ω là một tập con thực sự của không gian biến, và **không ràng buộc** (unconstrained) khi Ω trùng với toàn bộ không gian biến.

Hai thách thức lớn khiến tối ưu hóa trở thành một bài toán khó nói chung là:

- **Cực trị cục bộ và cực trị toàn cục.** Khi hàm mục tiêu không lồi (non-convex) hoặc đa cực trị (multimodal), một thuật toán chỉ dựa vào thông tin cục bộ quanh điểm hiện tại — ví dụ đạo hàm — có xu hướng hội tụ về cực trị cục bộ (local optimum) gần điểm khởi tạo nhất, thay vì cực trị toàn cục (global optimum) của toàn miền Ω . Đây chính là hạn chế cốt lõi của các phương pháp tối ưu dựa trên gradient khi áp dụng cho bài toán không lồi.
- **Bài toán NP-hard và sự bùng nổ tổ hợp.** Với nhiều lớp bài toán tối ưu tổ hợp (ví dụ bài toán người bán hàng — TSP, bài toán lập lịch), số lượng phương án khả thi tăng theo hàm mũ hoặc giai thừa của kích thước bài toán, khiến việc

duyet toàn bộ không gian tìm kiếm trở nên bất khả thi về mặt tính toán ngay cả với các bài toán kích thước vừa phải.

Hai thách thức trên là động lực chính cho sự phát triển của các phương pháp tìm kiếm không dựa trên đạo hàm, có khả năng khám phá đồng thời nhiều vùng của không gian tìm kiếm — trong đó có họ thuật toán tiến hóa được trình bày ở mục tiếp theo.

Họ Thuật toán Tiến hóa (Evolutionary Computation)

Khác với các phương pháp tối ưu cổ điển dựa trên đạo hàm (ví dụ Gradient Descent, phương pháp Newton) chỉ xử lý một điểm duy nhất tại mỗi bước lặp, **metaheuristic** là các chiến lược tìm kiếm ở mức tổng quát, không đảm bảo tìm được nghiệm tối ưu toàn cục tuyệt đối nhưng thường tìm được nghiệm đủ tốt trong thời gian chấp nhận được. Metaheuristic đặc biệt hữu ích khi hàm mục tiêu không khả vi, có nhiễu, không lồi, hoặc thậm chí không có biểu thức giải tích tường minh (black-box function).

Một nhánh quan trọng của metaheuristic là các **thuật toán tìm kiếm dựa trên quần thể** (population-based search): thay vì duy trì một điểm duy nhất, thuật toán duy trì một tập hợp nhiều điểm (quần thể) và để chúng cùng tiến hóa qua các thế hệ, nhờ đó khảo sát đồng thời nhiều vùng của không gian tìm kiếm và giảm nguy cơ mắc kẹt tại một cực trị cục bộ duy nhất. **Họ Thuật toán Tiến hóa** (Evolutionary Algorithms — EA) là đại diện tiêu biểu nhất của nhóm này, mô phỏng nguyên lý chọn lọc tự nhiên của Darwin để dẫn dắt quá trình tìm kiếm. Họ EA bao gồm bốn nhánh chính, phân biệt nhau chủ yếu ở cách biểu diễn giải pháp và toán tử tiến hóa được nhấn mạnh:

- **Thuật toán Di truyền (Genetic Algorithm — GA):** nhấn mạnh vai trò của lai ghép (crossover) như toán tử tìm kiếm chính, kết hợp thông tin từ nhiều cá thể cha mẹ để tạo ra cá thể con; đây là đối tượng nghiên cứu chính của chương này.
- **Chiến lược Tiến hóa (Evolution Strategies — ES):** do Rechenberg và Schwefel phát triển, tập trung vào tối ưu tham số thực với đột biến Gauss là toán tử chính và có cơ chế tự thích nghi (self-adaptation) các tham số đột biến trong quá trình tiến hóa.
- **Lập trình Tiến hóa (Evolutionary Programming — EP):** do Fogel đề xuất, tiến hóa chủ yếu thông qua đột biến trên biểu diễn hành vi (phenotype) của cá thể, gần như không sử dụng lai ghép.
- **Lập trình Di truyền (Genetic Programming — GP):** do Koza phát triển, mở rộng GA để tiến hóa trực tiếp các cấu trúc cây biểu thức hoặc chương trình

máy tính, thường dùng để tự động khám phá công thức toán học hoặc chương trình giải quyết một tác vụ cho trước.

Trong phạm vi tiểu luận này, trọng tâm là Thuật toán Di truyền áp dụng cho bài toán tối ưu số thực có ràng buộc, với cách mã hóa và các toán tử cụ thể được trình bày trong các mục tiếp theo.

1.2 Cơ sở Khoa học và Nguyên lý Hoạt động của GA

Nguồn gốc Lịch sử và Nguyên lý Sinh học

Thuật toán Di truyền được John Holland giới thiệu năm 1975 tại Đại học Michigan trong công trình *Adaptation in Natural and Artificial Systems* [1], và sau đó được David Goldberg hệ thống hoá, phổ biến rộng rãi hơn qua cuốn sách kinh điển năm 1989 *Genetic Algorithms in Search, Optimization, and Machine Learning* [2]. Ý tưởng nền tảng của GA là mô phỏng ba nguyên lý cốt lõi trong học thuyết tiến hóa của Darwin:

- **Chọn lọc tự nhiên** (“*Survival of the Fittest*”): cá thể thích nghi tốt hơn với môi trường có xác suất sống sót và sinh sản cao hơn.
- **Di truyền** (Heredity): đặc điểm của cá thể cha mẹ được truyền lại cho thế hệ con thông qua vật chất di truyền.
- **Biến dị** (Variation): các sai khác ngẫu nhiên xuất hiện giữa các cá thể — do lai ghép và đột biến — là nguồn tạo ra tính đa dạng cần thiết để quá trình chọn lọc có tác dụng.

GA chuyển ba nguyên lý sinh học trên thành một quy trình tính toán bằng cách xây dựng một phép tương ứng giữa các khái niệm sinh học và các thành phần thuật toán, được tóm tắt trong Bảng 1.1.

Nhờ phép tương ứng này, một bài toán tối ưu bất kỳ có thể được diễn dịch lại thành một quá trình “tiến hóa nhân tạo”: xuất phát từ một quần thể giải pháp ngẫu nhiên, để các giải pháp tốt hơn có nhiều cơ hội “sinh sản” hơn qua nhiều thế hệ, quần thể dần dịch chuyển về phía các vùng có giá trị hàm mục tiêu tốt của không gian tìm kiếm.

Phương pháp Mã hóa Giải pháp (Representation / Encoding)

Bước đầu tiên khi áp dụng GA cho một bài toán cụ thể là chọn cách biểu diễn (mã hóa) một giải pháp dưới dạng “nhiễm sắc thể” mà các toán tử di truyền có thể thao tác được. Bốn cách mã hóa phổ biến nhất là:

Bảng 1.1: Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền

Sinh học	Thuật toán Di truyền (GA)
Nhiễm sắc thể (Chromosome)	Mã hóa giải pháp (Solution encoding)
Gen (Gene)	Một biến / giá trị thành phần trong giải pháp
Cá thể (Individual)	Một giải pháp ứng viên
Quần thể (Population)	Tập hợp các giải pháp tại một thế hệ
Sức sống / Độ thích nghi	Giá trị hàm độ thích nghi (Fitness score)
Thế hệ (Generation)	Một vòng lặp tiến hóa

Mã hóa Nhị phân (Binary Encoding): giải pháp được biểu diễn bằng một chuỗi bit 0 và 1, ví dụ 10110010. Đây là cách mã hóa nguyên thủy nhất trong GA cổ điển của Holland, phù hợp với các bài toán rời rạc hoặc logic, nhưng khi áp dụng cho biến số thực cần một bước rời rạc hóa (lượng tử hóa) làm giảm độ chính xác.

Mã hóa Số thực (Real-value Encoding): giải pháp là một dãy số thực, ví dụ $[3.14, -0.05, 12.8]$, mỗi phần tử tương ứng trực tiếp với một biến quyết định. Cách mã hóa này phù hợp tự nhiên với các bài toán tối ưu hóa liên tục nhiều biến, tránh được sai số lượng tử hóa của mã hóa nhị phân, và là cách mã hóa được sử dụng trong phần cài đặt thực nghiệm của tiểu luận (Chương 2).

Mã hóa Hoán vị (Permutation Encoding): giải pháp là một thứ tự sắp xếp của một tập hợp phần tử, ví dụ $[3, 1, 4, 2, 5]$. Đây là cách mã hóa tự nhiên cho các bài toán định tuyến/lập lịch như bài toán người bán hàng (TSP) hay bài toán phân công công việc (Scheduling), nơi lời giải về bản chất là một hoán vị.

Mã hóa Dựa trên Cấu trúc (Tree/Graph Encoding): giải pháp được biểu diễn dưới dạng cây biểu thức hoặc đồ thị, sử dụng chủ yếu trong Lập trình Di truyền (Genetic Programming) để tiến hóa các công thức hoặc chương trình.

Việc lựa chọn cách mã hóa quyết định trực tiếp các toán tử lai ghép và đột biến nào có thể áp dụng được, như sẽ trình bày ở Mục 1.3.

1.3 Các Thành phần và Phép toán Cốt lõi

Hàm độ thích nghi (Fitness Function) và Ràng buộc

Hàm độ thích nghi (fitness function) đóng vai trò đánh giá chất lượng của từng cá thể trong quần thể, làm cơ sở cho phép chọn lọc. Với bài toán tối ưu không ràng buộc, hàm thích nghi thường được xây dựng trực tiếp từ hàm mục tiêu (objective function), ví

dự đảo dấu hàm mục tiêu khi bài toán gốc là bài toán cực tiểu hóa nhưng GA nguyên thủy được phát biểu dưới dạng cực đại hóa độ thích nghi.

Đối với bài toán **có ràng buộc** — trọng tâm của tiểu luận này — cách tiếp cận kinh điển là chuyển ràng buộc thành một số hạng phạt (penalty) cộng vào hàm mục tiêu, tạo thành hàm thích nghi đã phạt:

$$F(x) = f(x) \pm \sum_i \lambda_i \cdot g_i(x), \quad (1.2)$$

trong đó $g_i(x)$ đo mức độ vi phạm ràng buộc thứ i (bằng 0 nếu ràng buộc được thỏa mãn), còn $\lambda_i > 0$ là hệ số phạt tương ứng; dấu $+$ hay $-$ và độ lớn của λ_i được chọn tùy theo bài toán là cực tiểu hay cực đại hóa, sao cho cá thể vi phạm ràng buộc luôn bị đánh giá kém hơn cá thể khả thi có cùng giá trị hàm mục tiêu.

Hạn chế cố hữu của hàm phạt tĩnh (static penalty) là hệ số λ_i phải được chọn thủ công và cố định trong suốt quá trình tiến hóa: chọn λ_i quá nhỏ khiến quần thể “phớt lờ” ràng buộc, trong khi chọn quá lớn khiến số hạng phạt lấn át hoàn toàn hàm mục tiêu, làm quần thể chỉ tập trung thỏa mãn ràng buộc mà mất khả năng phân biệt các nghiệm khả thi tốt và xấu. Nhiều phương pháp xử lý ràng buộc nâng cao hơn đã được đề xuất để khắc phục hạn chế này — ví dụ quy tắc khả thi của Deb (feasibility rules) hay các ngưỡng ε giảm dần theo thời gian (Takahama và Sato) — và sẽ được trình bày cụ thể cùng phần cài đặt thực nghiệm ở Chương 2.

Các Phép chọn lọc (Selection Operators)

Phép chọn lọc quyết định cá thể nào trong quần thể hiện tại được chọn làm “bố mẹ” để sinh ra thế hệ tiếp theo, thiên vị có chủ đích về phía các cá thể có độ thích nghi cao hơn.

Vòng quay Roulette (Roulette Wheel Selection): xác suất chọn cá thể i tỉ lệ thuận với độ thích nghi f_i của nó so với tổng độ thích nghi toàn quần thể:

$$P_i = \frac{f_i}{\sum_{j=1}^N f_j}. \quad (1.3)$$

Hạn chế của phương pháp này là khi một cá thể có độ thích nghi vượt trội hẳn phần còn lại (super-individual), nó sẽ chiếm phần lớn xác suất được chọn, khiến quần thể nhanh chóng mất đa dạng.

Thách đấu (Tournament Selection): chọn ngẫu nhiên k cá thể từ quần thể, sau đó cá thể tốt nhất trong nhóm k cá thể này được chọn làm bố/mẹ. Tham số k (kích thước thách đấu) điều khiển trực tiếp áp lực chọn lọc: k càng lớn, áp lực

chọn lọc càng cao và quần thể hội tụ càng nhanh, nhưng cũng dễ mất đa dạng hơn.

Xếp hạng (Rank-based Selection): cá thể được chọn dựa trên thứ hạng của nó trong quần thể (sau khi sắp xếp theo độ thích nghi) thay vì giá trị thích nghi tuyệt đối, giúp tránh hiện tượng một cá thể siêu việt áp đảo toàn bộ xác suất chọn như ở Roulette Wheel.

Bên cạnh các toán tử chọn lọc bố mẹ nêu trên, một chiến lược lựa chọn cá thể sang thế hệ sau đặc biệt quan trọng là **giữ lại cá thể ưu tú (Elitism)**, được trình bày riêng ở phần dưới đây do vai trò và cách áp dụng của nó khác với chọn lọc bố mẹ thông thường.

Các Phép lai ghép (Crossover Operators)

Lai ghép (crossover) kết hợp vật chất di truyền của hai cá thể bố mẹ để tạo ra cá thể con, với tần suất lai ghép p_c (crossover rate) thường nằm trong khoảng 0.6 đến 0.95 — nghĩa là phần lớn, nhưng không phải toàn bộ, các cặp bố mẹ được chọn sẽ thực sự lai ghép; phần còn lại được sao chép nguyên vẹn sang thế hệ con.

Đối với mã hóa nhị phân và số thực, ba toán tử phổ biến nhất là:

- **Lai ghép đơn điểm (Single-point Crossover):** chọn ngẫu nhiên một vị trí cắt trên chuỗi nhiễm sắc thể, hai cá thể con được tạo bằng cách hoán đổi các đoạn nằm sau vị trí cắt giữa hai bố mẹ.
- **Lai ghép đa điểm (Multi-point Crossover):** tổng quát hóa lai ghép đơn điểm với nhiều vị trí cắt, các đoạn xen kẽ giữa hai bố mẹ được hoán đổi.
- **Lai ghép đồng nhất (Uniform Crossover):** mỗi vị trí gen được quyết định độc lập, ngẫu nhiên xem sẽ lấy từ bố hay mẹ, không phụ thuộc vào vị trí trong chuỗi.

Với mã hóa số thực, một họ toán tử quan trọng khác là lai ghép trộn (blend crossover, BLX- α) — cách tiếp cận được sử dụng trong phần cài đặt của tiểu luận. Với hai cá thể bố mẹ $p_1, p_2 \in \mathbb{R}^n$ và một hệ số trộn α được lấy mẫu ngẫu nhiên (có thể vượt ra ngoài khoảng $[0, 1]$), hai cá thể con được tạo theo tổ hợp tuyến tính:

$$c_1 = \alpha \cdot p_1 + (1 - \alpha) \cdot p_2, \quad c_2 = \alpha \cdot p_2 + (1 - \alpha) \cdot p_1. \quad (1.4)$$

Khi α được lấy mẫu từ một khoảng mở rộng ra ngoài $[0, 1]$ (ví dụ $\alpha \sim \mathcal{U}(-0.25, 1.25)$) thay vì giới hạn trong đoạn nối hai bố mẹ, cá thể con có thể nằm ngoài đoạn thẳng nối p_1 và p_2 , giúp toán tử này vừa có khả năng khai thác (exploitation) vùng lân cận

bố mẹ, vừa giữ được khả năng khám phá (exploration) ra ngoài đoạn đó thay vì chỉ co dần khoảng tìm kiếm qua các thế hệ.

Đối với mã hóa hoán vị (bài toán định tuyến/lập lịch), việc hoán đổi trực tiếp từng đoạn như trên có thể tạo ra cá thể con vi phạm tính hoán vị (một phần tử xuất hiện nhiều lần hoặc bị thiếu), nên cần các toán tử chuyên biệt:

- **PMX (Partially Matched Crossover)**: hoán đổi một đoạn con giữa hai bố mẹ như lai ghép đơn điểm, sau đó sửa các vị trí còn lại theo một ánh xạ tương ứng từng phần để đảm bảo tính hoán vị.
- **OX (Order Crossover)**: giữ nguyên một đoạn con từ bố mẹ thứ nhất, các phần tử còn lại được điền theo đúng thứ tự xuất hiện của chúng ở bố mẹ thứ hai.
- **CX (Cycle Crossover)**: xác định các “chu trình” (cycle) giữa vị trí phần tử ở hai bố mẹ, mỗi cá thể con được tạo bằng cách giữ nguyên các chu trình xen kẽ từ mỗi bố mẹ.

Các Phép đột biến (Mutation Operators)

Đột biến (mutation) tạo ra biến dị ngẫu nhiên trên một cá thể, với tần suất đột biến p_m (mutation rate) trên mỗi gen thường nhỏ, nằm trong khoảng 0.001 đến 0.05 đối với GA nhị phân cổ điển; vai trò chính của đột biến là duy trì đa dạng di truyền và giúp thuật toán thoát khỏi các vùng mà lai ghép một mình không thể tiếp cận được.

- **Đột biến đảo bit (Bit-flip Mutation)**: với mã hóa nhị phân, mỗi bit được đảo giá trị ($0 \leftrightarrow 1$) độc lập với xác suất p_m .
- **Đột biến Gauss / Uniform**: với mã hóa số thực, mỗi biến x_i được cộng thêm một nhiễu ngẫu nhiên, phổ biến nhất là nhiễu Gauss:

$$x'_i = x_i + \mathcal{N}(0, \sigma_i^2), \quad (1.5)$$

trong đó độ lệch chuẩn σ_i thường được đặt tỉ lệ với độ rộng miền giá trị của biến x_i , sao cho bước đột biến có ý nghĩa tương đối giống nhau giữa các biến có thang đo khác nhau.

- **Đột biến hoán đổi, đảo ngược, chèn** (Swap, Inversion, Insertion): dành cho mã hóa hoán vị — lần lượt là hoán đổi vị trí hai phần tử, đảo ngược thứ tự một đoạn con, và di chuyển một phần tử sang vị trí khác trong chuỗi — mỗi thao tác đều bảo toàn tính hoán vị của cá thể.

Chiến lược giữ lại Cá thể Ưu tú (Elitism)

Vì chọn lọc, lai ghép và đột biến đều mang tính ngẫu nhiên, không có gì đảm bảo cá thể tốt nhất của thế hệ hiện tại sẽ xuất hiện lại ở thế hệ kế tiếp — trên lý thuyết, độ thích nghi tốt nhất toàn quần thể hoàn toàn có thể giảm đi giữa hai thế hệ liên tiếp. **Elitism** khắc phục vấn đề này bằng cách bắt buộc giữ lại nguyên vẹn k cá thể tốt nhất của thế hệ hiện tại, chuyển thẳng sang thế hệ sau mà không qua lai ghép hay đột biến. Nhờ đó, độ thích nghi tốt nhất từng tìm được không bao giờ giảm qua các thế hệ (đơn điệu không giảm). Việc chọn kích thước elitism k là một sự đánh đổi: k quá nhỏ có nguy cơ đánh mất cá thể tốt vào tay các toán tử ngẫu nhiên, còn k quá lớn làm giảm áp lực khám phá của quần thể, có thể góp phần làm giảm đa dạng của quần thể và khiến quá trình tìm kiếm hội tụ sớm về một vùng hẹp của không gian tìm kiếm.

1.4 Quy trình Thuật toán và Điều kiện Dừng

Sơ đồ khối và Giả mã (Pseudocode)

Kết hợp các thành phần đã trình bày ở Mục 1.3, một thể hệ GA điển hình lặp lại chu trình: chọn lọc bố mẹ, lai ghép, đột biến, đánh giá quần thể con, rồi cập nhật quần thể mới có áp dụng elitism — như minh họa ở Hình 1.1.

Thuật toán 1 trình bày lại chu trình này dưới dạng giả mã.

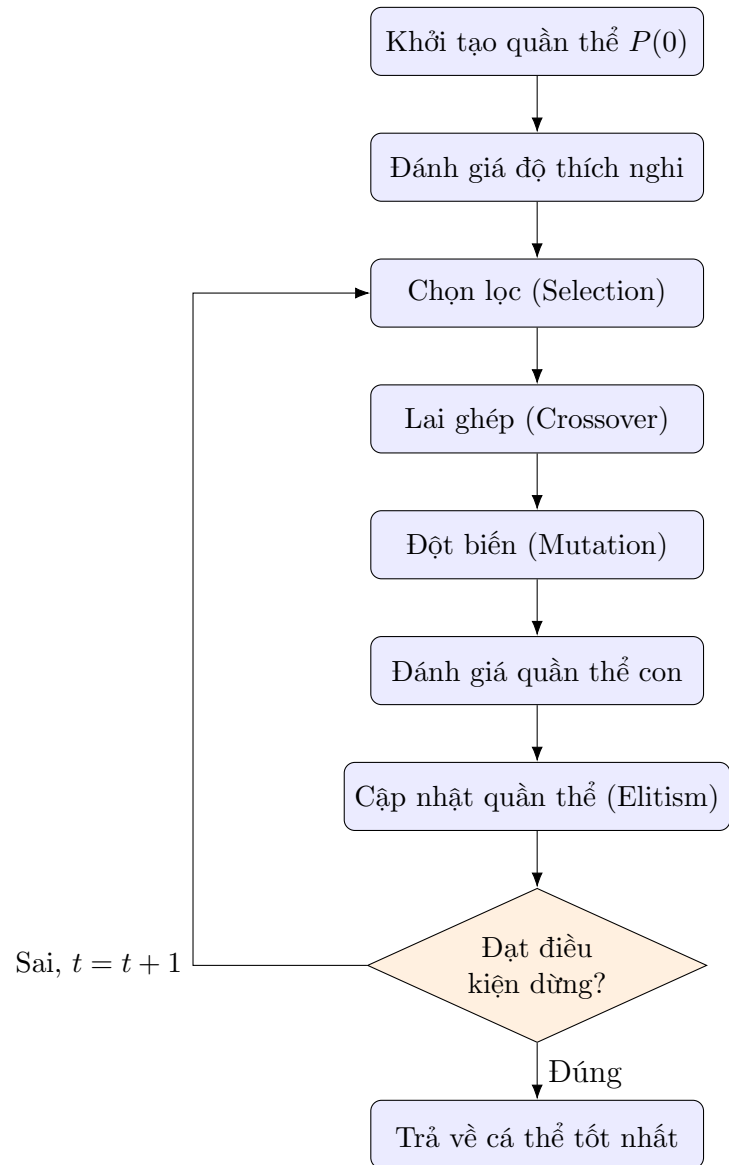
Thuật toán 1 Giải mã tổng quát của Thuật toán Di truyền

```
1:  $t \leftarrow 0$ 
2: Khởi tạo quần thể ban đầu  $P(0)$  ngẫu nhiên
3: Đánh giá độ thích nghi của  $P(0)$ 
4: while điều kiện dừng chưa thỏa mãn do
5:    $t \leftarrow t + 1$ 
6:   Chọn lọc tập bố mẹ  $S(t)$  từ  $P(t - 1)$ 
7:   Thực hiện lai ghép (crossover) trên  $S(t)$ , tạo tập con  $C(t)$ 
8:   Thực hiện đột biến (mutation) trên  $C(t)$ 
9:   Đánh giá độ thích nghi của  $C(t)$ 
10:   $P(t) \leftarrow$  cập nhật quần thể mới (áp dụng elitism)
11: end while
12: return cá thể tốt nhất tìm được
```

Tiêu chuẩn Dừng và Đánh giá sự Hội tụ

Vòng lặp tiến hóa cần một điều kiện dừng (termination criterion) tường minh; ba tiêu chuẩn phổ biến, có thể kết hợp với nhau, là:

- **Đạt số lượng thế hệ tối đa G_{max} :** đơn giản và dễ kiểm soát ngân sách tính toán nhất, nhưng G_{max} quá nhỏ có nguy cơ dừng trước khi quần thể hội tụ, trong



Hình 1.1: Chu trình tiến hóa của Thuật toán Di truyền

khi G_{max} quá lớn gây lãng phí tính toán không cần thiết sau khi quần thể đã hội tụ.

- **Đạt giá trị fitness mục tiêu:** dừng ngay khi tìm được một cá thể có độ thích nghi đạt hoặc vượt một ngưỡng đặt trước — phù hợp khi đã biết (hoặc ước lượng được) giá trị tối ưu cần đạt.
- **Quần thể hội tụ (stagnation):** dừng khi độ thích nghi của cá thể tốt nhất không cải thiện sau K thế hệ liên tiếp — dấu hiệu cho thấy các toán tử di truyền không còn tạo ra tiến triển đáng kể. Cần lưu ý tiêu chuẩn này có thể dừng thuật toán quá sớm nếu K quá nhỏ, do quần thể có thể dừng cải thiện tạm thời trước khi tiếp tục tiến triển ở các thế hệ sau.

1.5 Lập trình Di truyền (Genetic Programming)

Lập trình Di truyền (Genetic Programming — GP), do John Koza phát triển từ cuối thập niên 1980 và hệ thống hoá trong công trình kinh điển năm 1992 [8], là một mở rộng tự nhiên của GA đã giới thiệu ở Mục 1.2 dưới tên gọi *mã hóa dựa trên cấu trúc*. Khác biệt cốt lõi so với GA mã hóa số thực hay hoán vị là: thay vì tiến hóa một chuỗi giá trị có độ dài cố định trong một khuôn dạng lời giải cho trước, GP tiến hóa trực tiếp **cấu trúc** của một biểu thức hoặc chương trình, cho phép thuật toán tự khám phá đồng thời cả hình dạng lẫn tham số của lời giải.

Mã hóa dạng cây biểu thức

Mỗi cá thể trong GP là một cây biểu thức (expression tree): nút trong (internal node) là một toán tử lấy từ một *tập hàm* (function set) cho trước — ví dụ các phép toán số học $\{+, -, \times, \div\}$ hay hàm lượng giác $\{\sin, \cos\}$ — còn nút lá (terminal node) là một biến đầu vào hoặc một hằng số, lấy từ một *tập đầu cuối* (terminal set). Duyệt cây theo đúng cấu trúc phân cấp của nó cho ra một biểu thức toán học tường minh, tính được giá trị trên bất kỳ bộ dữ liệu đầu vào nào — nhờ đó, một quần thể các cây khác nhau về cả hình dạng lẫn nội dung có thể được đánh giá và so sánh với nhau như trong GA thông thường.

Hàm độ thích nghi

Bài toán tiêu biểu nhất mà GP được áp dụng là **hồi quy ký hiệu** (symbolic regression): tự động tìm một biểu thức toán học mô tả tốt một tập dữ liệu quan sát (X, y) mà không giả định trước dạng hàm cụ thể — khác hẳn hồi quy tuyến tính hay hồi quy đa thức, nơi dạng hàm đã được ấn định sẵn và chỉ các hệ số được tối ưu. Độ thích nghi của một cây trong bài toán này thường được xây dựng từ sai số dự đoán, ví dụ sai

số bình phương trung bình (MSE) giữa giá trị cây tính ra và giá trị y thực tế, cộng thêm một số hạng phạt tỉ lệ với kích thước cây (*parsimony pressure*) nhằm hạn chế hiện tượng cây “phình to” không kiểm soát (bloat) mà không mang lại cải thiện tương xứng về độ chính xác.

Toán tử tiến hóa

Lai ghép và đột biến trong GP thao tác trực tiếp trên cấu trúc cây thay vì trên một chuỗi có độ dài cố định:

- **Lai ghép trao đổi cây con (Subtree Crossover):** chọn ngẫu nhiên một nút trên mỗi cây bố mẹ, rồi hoán đổi hai cây con bắt rễ tại các nút đó với nhau để tạo ra hai cây con.
- **Đột biến thay cây con (Subtree Mutation):** chọn ngẫu nhiên một nút trên cây, rồi thay toàn bộ cây con bắt rễ tại nút đó bằng một cây con mới được sinh ngẫu nhiên.

Các thành phần còn lại của chu trình tiến hóa — khởi tạo quần thể, chọn lọc, elitism, điều kiện dừng — về bản chất giống hệt GA đã trình bày ở các mục trước, chỉ khác đối tượng thao tác là một **cây** thay vì một **vector** số hay hoán vị.

Trên cơ sở các thành phần và quy trình đã trình bày ở chương này, Chương 2 sẽ trình bày cụ thể việc áp dụng GA (và GP) cho bốn nhóm bài toán — tối ưu không ràng buộc, tối ưu có ràng buộc, bài toán người du lịch, và hồi quy ký hiệu — đồng thời so sánh thực nghiệm với các phương pháp có sẵn tương ứng (PyGAD, SciPy SLSQP, nghiệm tối ưu công bố, hồi quy tuyến tính). Độc giả quan tâm đến các hướng phát triển và ứng dụng khác của GA có thể tham khảo thêm tổng quan của Katoch và cộng sự (2020) [5], cũng như một ứng dụng kinh điển kết hợp GA với huấn luyện mạng nơ-ron của Whitley và cộng sự (1990) [6].

Chương 2

Ứng dụng: Thực nghiệm trên Bốn Bài toán

Chương này trình bày việc áp dụng các nguyên lý đã trình bày ở Chương 1 vào bốn nhóm bài toán cụ thể. Mỗi nhóm sử dụng một cách mã hóa khác nhau: tối ưu không ràng buộc dùng mã hóa số thực (Mục 2.1), tối ưu có ràng buộc cũng dùng mã hóa số thực nhưng kết hợp thêm cơ chế xử lý ràng buộc (Mục 2.2), bài toán người du lịch dùng mã hóa hoán vị (Mục 2.3), và hồi quy ký hiệu dùng mã hóa dạng cây biểu thức theo nguyên lý Lập trình Di truyền (Mục 2.4). Với mỗi bài toán, phát biểu toán học đầy đủ được trình bày trước, sau đó là phương pháp so sánh được sử dụng và bảng kết quả thực nghiệm.

2.1 Tối ưu không ràng buộc: So sánh GA với Py-GAD

Năm hàm benchmark kinh điển của tối ưu không ràng buộc được sử dụng để đánh giá khả năng tìm cực trị toàn cục của GA. Cả năm hàm đều có hai biến x, y và một cực tiểu toàn cục đã biết trước, cho phép so sánh trực tiếp giá trị GA tìm được với giá trị đúng.

Hàm Easom có công thức

$$f(x, y) = -\cos(x) \cos(y) \exp(-(x - \pi)^2 - (y - \pi)^2), \quad x, y \in [-100, 100], \quad (2.1)$$

với cực tiểu toàn cục $f(\pi, \pi) = -1$. Giá trị hàm gần như bằng 0 trên toàn bộ miền tìm kiếm, ngoại trừ một hố hẹp duy nhất chứa cực tiểu toàn cục — vì miền tìm kiếm rộng $([-100, 100]^2)$ so với kích thước hố cực tiểu, phần lớn không gian tìm kiếm không cung cấp thông tin nào để dẫn đường, nên hàm này kiểm tra khả năng *tìm ra* đúng vùng chứa nghiệm của một thuật toán tìm kiếm toàn cục.

Hàm Rastrigin có công thức

$$f(x, y) = 20 + x^2 + y^2 - 10 \cos(2\pi x) - 10 \cos(2\pi y), \quad x, y \in [-5.12, 5.12], \quad (2.2)$$

với cực tiểu toàn cục $f(0, 0) = 0$. Hai số hạng cosin tạo ra rất nhiều cực tiểu cục bộ phân bố đều và dày đặc xung quanh cực tiểu toàn cục, nên hàm này kiểm tra khả năng không bị mắc kẹt tại một trong các cực tiểu cục bộ đó.

Hàm Rosenbrock có công thức

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad x \in [-5, 10], \quad y \in [-5, 10], \quad (2.3)$$

với cực tiểu toàn cục $f(1, 1) = 0$. Hàm chỉ có một cực tiểu duy nhất, nhưng cực tiểu đó nằm trong một thung lũng cong hẹp hình quả chuối: khó khăn ở đây không phải là tìm ra vùng chứa nghiệm mà là dò theo chính xác đáy của thung lũng cong này.

Hàm Ackley có công thức

$$f(x, y) = -20 \exp\left(-0.2\sqrt{0.5(x^2 + y^2)}\right) - \exp(0.5 \cos(2\pi x) + 0.5 \cos(2\pi y)) + 20 + e, \quad (2.4)$$

với $x, y \in [-32.768, 32.768]$ và cực tiểu toàn cục $f(0, 0) = 0$. Về tổng thể hàm có dạng một hố lớn duy nhất, nhưng số hạng cosin tạo thêm vô số gợn sóng cực tiểu cục bộ nhỏ phủ lên bề mặt hố đó.

Hàm Schwefel có công thức

$$f(x, y) = 837.9658 - x \sin \sqrt{|x|} - y \sin \sqrt{|y|}, \quad x, y \in [-500, 500], \quad (2.5)$$

với cực tiểu toàn cục $f(420.9687, 420.9687) \approx 0$. Đây là hàm được thiết kế để đánh lừa các thuật toán: vùng lân cận gốc tọa độ có giá trị hàm nhỏ và trông giống một cực tiểu, nhưng cực tiểu toàn cục thực sự lại nằm cách xa gốc tọa độ, gần biên của miền tìm kiếm.

Với mỗi hàm, GA mã hóa số thực (lai ghép trộn BLX- α , đột biến Gauss, theo Mục 1.3) được chạy độc lập ba lần với quần thể 500 cá thể trong tối đa 50 thế hệ. Đối trọng so sánh là PyGAD, một thư viện GA có sẵn, được chạy trên cùng bài toán, cùng tham số quần thể/thế hệ, và được cấu hình dùng đúng công thức lai ghép trộn và đột biến Gauss của GA tự cài — nhờ đó phần chênh lệch kết quả (nếu có) phản ánh đúng sự khác biệt về kiến trúc vòng lặp tiến hóa giữa hai cách cài đặt, chứ không phải khác biệt về công thức toán học của từng toán tử. Kết quả tốt nhất trong ba lần chạy của mỗi phương pháp được trình bày ở Bảng 2.1.

Cả hai phương pháp đều tìm đúng, hoặc gần đúng đến sai số không đáng kể, cực tiểu toàn cục trên cả năm hàm — kể cả Easom và Schwefel, hai hàm được thiết kế

Bảng 2.1: Kết quả GA và PyGAD trên năm hàm benchmark không ràng buộc

Hàm	Nghiệm đúng	GA (f^*)	PyGAD (f^*)
Easom	-1	-1.0000000000	-1.0000000000
Rastrigin	0	0.0000000000	0.0000000000
Rosenbrock	0	0.0001641030	0.0006899893
Ackley	0	5.9089×10^{-11}	2.9921×10^{-11}
Schwefel	≈ 0	2.5455×10^{-5}	2.5455×10^{-5}

riêng để đánh lừa các thuật toán chỉ khai thác thông tin cục bộ quanh điểm hiện tại. Kết quả này minh chứng trực tiếp cho ưu điểm cốt lõi của tìm kiếm dựa trên quần thể đã trình bày ở Mục 1.1: vì GA và PyGAD duy trì nhiều cá thể trải rộng trên không gian tìm kiếm thay vì một điểm duy nhất, cả hai đều không phụ thuộc vào việc điểm khởi tạo có nằm gần cực tiểu toàn cục hay không, khác với các phương pháp gradient cổ điển. Riêng sai số ở Rosenbrock lớn hơn hẳn bốn hàm còn lại ở cả hai phương pháp; điều này phù hợp với đặc điểm địa hình đã nêu ở trên, vì một khi quần thể đã lọt được vào đúng thung lũng, tốc độ hội tụ tiếp theo phụ thuộc vào khả năng dò theo một đường cong hẹp — một việc mà toán tử lai ghép/đột biến ngẫu nhiên đẳng hướng không làm tốt bằng việc định vị một vùng rộng ban đầu.

2.2 Tối ưu có ràng buộc: So sánh GA với SciPy SLSQP

Năm bài toán tối ưu có ràng buộc được lựa chọn để bao quát các dạng ràng buộc và tính chất hàm mục tiêu khác nhau, từ tuyến tính tuyệt đối đến phi tuyến.

Bài toán 1 — Entropy tối đa. Đây là bài toán “con xúc xắc bị làm lệch” của Jaynes: tìm phân phối xác suất $x_1, \dots, x_6$ của một con xúc xắc sáu mặt có entropy lớn nhất, biết trước kỳ vọng $E[X] = 4.5$. Sau khi đổi dấu để đưa về bài toán cực tiểu hóa (vì $\max H(x) \equiv \min -H(x)$), phát biểu toán học là

$$\min_{x \in \mathbb{R}^6} f(x) = \sum_{i=1}^6 x_i \ln x_i \quad \text{s.t.} \quad \sum_{i=1}^6 x_i = 1, \quad \sum_{i=1}^6 i x_i = 4.5, \quad x_i \geq 0. \quad (2.6)$$

Hai ràng buộc đẳng thức phải được thỏa mãn đồng thời, khiến miền khả thi chỉ còn là giao tuyến của hai siêu phẳng trong $\mathbb{R}^6$.

Bài toán 2 — Điểm gần nhất trong miền ràng buộc. Cho một điểm $(5, 4, -2)$ nằm ngoài miền ràng buộc, tìm điểm (x, y, z) trong miền ràng buộc gần điểm đó nhất theo khoảng cách Euclid bình phương — tức là bài toán chiếu một điểm lên một tập

lỗi:

$$\min_{x,y,z} f(x,y,z) = (x-5)^2 + (y-4)^2 + (z+2)^2 \quad \text{s.t.} \quad x+2y+z=4, \quad x^2+y^2 \leq 9, \quad z \geq 0. \quad (2.7)$$

Miền ràng buộc là giao của một mặt phẳng, một hình trụ, và một nửa không gian.

Bài toán 3 — Quy hoạch tuyến tính. Hàm mục tiêu và mọi ràng buộc đều tuyến tính:

$$\max_{x,y,z} f(x,y,z) = 3x + 5y - 2z \quad \text{s.t.} \quad x+2y+z=10, \quad 2x+y \leq 8, \quad x,y,z \geq 0. \quad (2.8)$$

Vì là quy hoạch tuyến tính (LP) thuần túy, nghiệm tối ưu chắc chắn nằm tại một đỉnh của miền đa diện lỗi xác định bởi các ràng buộc.

Bài toán 4 — Quy hoạch bình phương. Hàm mục tiêu bậc hai lỗi, ràng buộc tuyến tính:

$$\min_{x,y,z} f(x,y,z) = x^2 + 2y^2 + z^2 - 4x - 6y \quad \text{s.t.} \quad x+y+2z=5, \quad x+y \leq 3, \quad x,y,z \geq 0. \quad (2.9)$$

Vì hàm mục tiêu lỗi và miền ràng buộc lỗi, bài toán này (quy hoạch bình phương — QP) có đúng một cực tiểu toàn cục.

Bài toán 5 — Tối ưu lỗi phi tuyến. Hàm mục tiêu là tổng hai hàm mũ, khác hẳn tính chất tuyến tính từng khúc của bốn bài toán trên:

$$\min_{x,y} f(x,y) = e^{-x} + e^{-2y} \quad \text{s.t.} \quad 2x+3y=6, \quad x^2+y^2 \leq 4, \quad x,y \geq 0. \quad (2.10)$$

Cả hàm mục tiêu lẫn miền ràng buộc đều lỗi nên bài toán vẫn có đúng một cực tiểu toàn cục, dù không còn là dạng tuyến tính hay bình phương.

GA cho cả năm bài toán này sử dụng chiến lược xử lý ràng buộc đã giới thiệu ở Mục 1.3 — kết hợp quy tắc khả thi Deb với ngưỡng ε giảm dần theo thời gian, thay vì hàm phạt tĩnh — chạy với quần thể 1000 cá thể trong tối đa 500 thế hệ, lặp lại ba lần độc lập. Đối trọng so sánh là SciPy SLSQP, một phương pháp tối ưu dựa trên gradient giải theo điều kiện KKT, được chạy từ 30 điểm khởi tạo ngẫu nhiên khác nhau để giảm rủi ro hội tụ về một cực tiểu cục bộ thay vì cực tiểu toàn cục. Kết quả tốt nhất của mỗi phương pháp được trình bày ở Bảng 2.2.

SLSQP đạt kết quả nhanh hơn GA một chút ở cả năm bài toán. Điều này đúng như kỳ vọng lý thuyết, vì cả năm bài toán đều lỗi và có hàm mục tiêu khả vi — chính là điều kiện lý tưởng nhất cho một phương pháp dựa trên gradient và điều kiện KKT như SLSQP, trong khi GA không khai thác được thông tin đạo hàm này để tinh chỉnh nghiệm ở bước cuối. Tuy vậy, khoảng cách giữa hai phương pháp rất nhỏ: GA đạt trong khoảng 0.01% đến 1% so với SLSQP ở bốn trong năm bài toán; riêng bài toán

Bảng 2.2: Kết quả GA và SciPy SLSQP trên năm bài toán tối ưu có ràng buộc

Bài toán	GA (f^*)	SLSQP (f^*)
1. Entropy tối đa	-1.5985825694	-1.6135810982
2. Điểm gần nhất	20.3497958340	20.2756044350
3. Quy hoạch tuyến tính (LP)	-25.9149581523	-26.0000000000
4. Quy hoạch bình phương (QP)	-7.3329937494	-7.3333333333
5. Tối ưu lồi phi tuyến	0.3565975317	0.3565154830

3 (LP) có sai số tương đối lớn hơn một chút, vì nghiệm tối ưu của LP luôn nằm đúng tại một đỉnh sắc cạnh của miền đa diện — một điểm mà các toán tử lai ghép/đột biến liên tục của GA khó tiệm cận chính xác tuyệt đối hơn so với việc dò theo một đường cong trơn. Nhìn chung, kết quả cho thấy GA vẫn xấp xỉ tốt cả năm bài toán dù không dùng đến đạo hàm. Điều này có ý nghĩa thực tiễn quan trọng: trong các bài toán mà hàm mục tiêu không khả vi hoặc không lồi — nơi SLSQP không còn đảm bảo hội tụ đúng — khả năng xấp xỉ tốt của GA mà không cần thông tin đạo hàm trở thành một lợi thế thực sự, như sẽ thấy rõ hơn ở hai bài toán tiếp theo.

2.3 Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ

Bài toán người du lịch (Traveling Salesman Problem — TSP) được phát biểu như sau: cho n địa điểm và ma trận khoảng cách $D = (d_{ij})_{n \times n}$ giữa mọi cặp địa điểm, tìm một hoán vị $\pi = (\pi_1, \dots, \pi_n)$ của n địa điểm sao cho tổng độ dài lộ trình khép kín đi qua tất cả các địa điểm đúng một lần là nhỏ nhất:

$$\min_{\pi} L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1}. \quad (2.11)$$

Bài toán này được mã hóa dưới dạng hoán vị đã giới thiệu ở Mục 1.2: mỗi cá thể của GA chính là một hoán vị π , dùng lai ghép Order Crossover (OX) và đột biến Swap (Mục 1.3) thay cho các toán tử số thực ở hai mục trước.

Để đánh giá khách quan trên dữ liệu thực thay vì các ví dụ tự tạo quy mô nhỏ, ba bộ dữ liệu chuẩn từ thư viện benchmark TSPLIB95 [9] được sử dụng: berlin52 ($n = 52$ địa điểm tại Berlin), rd100 ($n = 100$ địa điểm ngẫu nhiên), và ch150 ($n = 150$ địa điểm). Mỗi bộ dữ liệu đều có kèm theo lộ trình tối ưu đã được công bố, dùng làm mốc đối chiếu khách quan thay cho phương pháp vét cạn — vốn cần thử $(n-1)!$ hoán vị và trở nên bất khả thi về mặt tính toán ngay từ n vài chục.

Thử nghiệm ban đầu với GA hoán vị thuần túy (chỉ dùng OX và Swap) cho kết

quả chênh lệch rất lớn so với nghiệm tối ưu đã công bố, từ 20% đến 84% tùy bộ dữ liệu và tăng dần theo số địa điểm n . Nguyên nhân là hai toán tử OX và Swap không có cơ chế sửa các đoạn đường bị cắt chéo nhau trong lộ trình — một lộ trình càng có nhiều địa điểm thì càng dễ chứa những đoạn cắt chéo như vậy, và OX/Swap chỉ có thể tạo ra một cấu hình mới ngẫu nhiên chứ không chủ động “gỡ” đoạn cắt chéo đó. Để khắc phục hạn chế này, một chiến lược lai giữa GA và tìm kiếm cục bộ (*memetic algorithm*) được áp dụng: sau mỗi thế hệ, một tỉ lệ cá thể tốt nhất trong quần thể được tinh chỉnh thêm bằng phép tìm kiếm cục bộ **2-opt** — xét lần lượt từng cặp cạnh trong lộ trình, hoán đổi hai cạnh đó nếu việc hoán đổi làm giảm tổng độ dài — trước khi tiếp tục vòng lặp tiến hóa (chọn lọc, lai ghép, đột biến) như bình thường. Với tỉ lệ tinh chỉnh bằng khoảng 5% quần thể mỗi thế hệ, cả ba bộ dữ liệu đều đạt đúng lộ trình tối ưu đã công bố, như trình bày ở Bảng 2.3.

Bảng 2.3: Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95

Bộ dữ liệu	Số địa điểm (n)	Nghiệm tối ưu	GA	Đạt được ở thế hệ
berlin52	52	7542	7542	8/50
rd100	100	7910	7910	48/50
ch150	150	6528	6528	37/50

Trong quá trình hiệu chỉnh tham số, một quan sát quan trọng được ghi nhận: tỉ lệ cá thể được tinh chỉnh bằng 2-opt cần được đặt theo *phần trăm* của quần thể chứ không phải theo một số lượng cố định. Cụ thể, khi giữ nguyên số lượng cá thể được tinh chỉnh ở một con số tuyệt đối nhưng tăng kích thước quần thể lên, kết quả trở nên tệ hơn so với quần thể nhỏ — nguyên nhân là vì trong một quần thể lớn, số cá thể đã qua tinh chỉnh chiếm tỉ lệ quá nhỏ nên khó lan gen tốt ra phần còn lại của quần thể thông qua phép chọn lọc. Quan sát này cho thấy các tham số của một chiến lược lai giữa GA và tìm kiếm cục bộ không thể được chọn độc lập với nhau, mà phải cân chỉnh tương thích với quy mô quần thể.

2.4 Hồi quy Ký hiệu bằng Lập trình Di truyền

Ứng dụng cuối cùng minh họa Lập trình Di truyền (Mục 1.5) cho bài toán hồi quy ký hiệu trên dữ liệu vật lý thực: các phép đo mật độ hạt, vận tốc, và từ trường của gió mặt trời do vệ tinh WIND của NASA ghi nhận. Ba thành phần từ trường đo được là B_X, B_Y, B_Z , và mật độ hạt đo được là n . Từ bốn đại lượng này, hai đại lượng vật lý phái sinh được chọn làm mục tiêu dự đoán cho GP:

$$|B|^2 = B_X^2 + B_Y^2 + B_Z^2 \quad (\text{bình phương độ lớn từ trường}), \quad (2.12)$$

$$V_A^2 \propto \frac{|B|^2}{n} \quad (\text{vận tốc Alfvén bình phương}). \quad (2.13)$$

Đại lượng thứ nhất là một quan hệ tổng bình phương (dạng Pythagore) giữa ba biến đo được; đại lượng thứ hai kết hợp thêm một phép chia cho biến mật độ hạt.

Điểm mấu chốt khiến hai đại lượng này trở thành phép thử công bằng cho GP là: quan hệ tổng bình phương ở Phương trình (2.12) *không thể* tuyến tính hóa bằng phép biến đổi logarit như các quan hệ dạng lũy thừa thông thường, vì B_X, B_Y, B_Z có thể mang giá trị âm nên không lấy logarit được. Do đó, Hồi quy Tuyến tính áp dụng trực tiếp trên ba đặc trưng thô B_X, B_Y, B_Z gần như chắc chắn không thể biểu diễn được cấu trúc bậc hai này, vì mô hình tuyến tính chỉ có thể biểu diễn một tổ hợp tuyến tính của chính các đặc trưng đó, không tự sinh ra được số hạng bậc hai. Ngược lại, GP có thể tự xây dựng số hạng bậc hai bằng cách nhân một biến với chính nó ngay trong cây biểu thức, mà không cần được cung cấp trước các đặc trưng B_X^2, B_Y^2, B_Z^2 như một phép biến đổi thủ công. Kết quả đối chiếu hai phương pháp trên tập kiểm tra được trình bày ở Bảng 2.4.

Bảng 2.4: Kết quả Hồi quy Tuyến tính và GP trên hai đại lượng vật lý phái sinh

Đại lượng	Phương pháp	MAE	RMSE	R^2
$ B ^2$	Hồi quy Tuyến tính	15.1284	19.9825	0.4784
	GP (hồi quy ký hiệu)	4.1820	6.0779	0.9517
V_A^2	Hồi quy Tuyến tính	2.7376	3.6804	0.4629
	GP (hồi quy ký hiệu)	1.0554	1.5486	0.9049

Kết quả cho thấy chênh lệch rõ rệt và nhất quán trên cả hai đại lượng. Hồi quy Tuyến tính chỉ giải thích được khoảng 46% đến 48% phương sai của dữ liệu ($R^2 \approx 0.46\text{--}0.48$), trong khi GP đạt trên 90% ($R^2 \approx 0.90\text{--}0.95$); đồng thời sai số dự đoán (MAE, RMSE) của GP chỉ còn chưa đến một phần ba so với Hồi quy Tuyến tính ở cả hai đại lượng. Đây là kết quả trái ngược hẳn với các bài toán hồi quy dạng lũy thừa từng được thử nghiệm trong quá trình thực hiện tiểu luận — ví dụ định luật Kepler III hay áp suất động của gió mặt trời — là những quan hệ trở thành tuyến tính tuyệt đối sau khi biến đổi logarit, khiến Hồi quy Tuyến tính ở các bài toán đó luôn đạt hoặc vượt GP. Sự tương phản giữa hai nhóm kết quả khẳng định đúng giả thuyết ban đầu: lợi thế thực sự của GP so với hồi quy cổ điển chỉ bộc lộ rõ khi quan hệ cần khám phá có cấu trúc phi tuyến mà phép biến đổi đặc trưng thủ công thông thường, như logarit, không xử lý được.

2.5 Tổng kết Chương

Bốn thực nghiệm trên minh họa một đặc điểm xuyên suốt của Thuật toán Di truyền: tính linh hoạt cao nhờ khả năng thích ứng với nhiều cách mã hóa khác nhau — số thực ở Mục 2.1 và 2.2, hoán vị ở Mục 2.3, và cây biểu thức ở Mục 2.4. Đôi lại tính linh hoạt đó là hiệu quả thấp hơn các phương pháp chuyên dụng khi bài toán có sẵn một cấu trúc mà phương pháp đó khai thác được: SLSQP khai thác tính khả vi và tính lồi của năm bài toán ở Mục 2.2. Ngược lại, ở những bài toán mà cấu trúc đó không có sẵn, hoặc phương pháp cổ điển tương ứng không còn đủ mạnh — bài toán người du lịch quy mô lớn cần bổ sung tìm kiếm cục bộ mới đạt được nghiệm tối ưu, hay quan hệ phi tuyến dạng tổng bình phương mà Hồi quy Tuyến tính không thể biểu diễn được — GA và GP thể hiện rõ giá trị thực tiễn của một phương pháp tìm kiếm không đặt giả định trước về cấu trúc bài toán.

Tài liệu tham khảo

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT Press.
4. Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
5. Katoch, S., Chauhan, S. S., & Kumar, V. (2020). A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications*, 80, 8091–8126.
6. Whitley, D., Starkweather, T., & Bogart, C. (1990). Genetic algorithms and neural networks: optimizing connections and connectivity. *Parallel Computing*, 14, 347–361.
7. Takahama, T., & Sato, S. (2010). Constrained optimization by the ε constrained differential evolution with gradient-based mutation and feasible elites. *IEEE Congress on Evolutionary Computation (CEC)*.
8. Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.
9. Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376–384.